

# Automatisation des Déploiements Multi-Cloud avec Terraform

---



# Plan de la présentation

---

- **Introduction générale**
- **Problématique du projet**
- **Généralités sur le cloud computing**
- **Déploiement des infrastructures cloud**
- **Infrastructure as Code (IaC)**
- **Présentation de Terraform**
- **Architecture et fonctionnement de Terraform**
- **Automatisation des déploiements avec Terraform**
- **Étude de cas pratique**
- **Conclusion et perspective**

# INTRODUCTION GÉNÉRALE

---

Le cloud computing permet d'utiliser des ressources informatiques via Internet sans gérer de matériel. Aujourd'hui, les entreprises utilisent plusieurs fournisseurs cloud, ce qu'on appelle le multi-cloud, afin de gagner en flexibilité et d'éviter la dépendance à un seul fournisseur. Cependant, la gestion manuelle de ces environnements est complexe et source d'erreurs. C'est pourquoi notre projet porte sur l'automatisation des déploiements multi-cloud avec Terraform, un outil qui permet de gérer les infrastructures de manière automatique et fiable.



# **Problématique du projet**

- La gestion manuelle des infrastructures cloud est longue et sujette aux erreurs.
- Chaque fournisseur (AWS, Azure, GCP) a ses propres outils et interfaces, ce qui complique le déploiement.
- Les erreurs humaines peuvent provoquer pannes, incohérences ou failles de sécurité.
- Suivre et maintenir des infrastructures manuellement est difficile et coûteux.

# GÉNÉRALITÉS SUR LE CLOUD

---

## COMPUTING

Le cloud computing est défini comme un modèle de fourniture de services informatiques sur Internet, où les utilisateurs peuvent accéder à des ressources partagées et configurables à la demande, sans avoir à gérer directement l'infrastructure physique.

**Exemple :** utilisation de Google Drive, AWS (Amazon Web Services), Microsoft Azure ou GCP (Google Cloud Platform).





# TYPES DE

---

Les clouds peuvent être classés selon leur mode de déploiement et d'accès :

## **Cloud public :**

- Propriétaire par un fournisseur externe (AWS, Azure, GCP).
- Accessible à tout utilisateur via Internet.
- Idéal pour les entreprises qui veulent réduire les coûts d'infrastructure.

## **Cloud privé :**

- Utilisé exclusivement par une seule organisation.
- Plus sécurisé et personnalisable.
- Nécessite des ressources internes ou un hébergement dédié.

# TYPES DE

---

## CLOUD

### **Cloud hybride :**

- Combine cloud public et privé.
- Permet de garder certaines données sensibles sur le cloud privé et d'utiliser le cloud public pour les tâches moins critiques.

### **Multi-cloud :**

- Utilisation de plusieurs fournisseurs de cloud simultanément.
- Réduit les risques de dépendance à un seul fournisseur.
- Optimise les coûts et les performances selon les besoins.



# Avantages

---

- Flexibilité et scalabilité : augmentation ou diminution des ressources selon les besoins.
  - Réduction des coûts : pas besoin d'investir dans des infrastructures lourdes.
  - Accessibilité : accès aux services et données depuis n'importe où avec Internet.
- Maintenance simplifiée : le fournisseur gère les mises à jour et la sécurité.





## Limites

- Sécurité et confidentialité : les données sont hébergées sur des serveurs externes.
  - Dépendance au fournisseur : risques de verrouillage ou d'interruption du service.
- Performance variable : dépend de la qualité de la connexion Internet.



# Déploiement des infrastructures cloud

Le déploiement d'infrastructures cloud consiste à mettre en place et configurer des ressources (serveurs, stockage, bases de données, réseaux, applications) pour qu'elles soient prêtes à l'usage. Il existe deux approches principales : **manuel** et **automatisé**.



# Déploiement des infrastructures cloud

## **Déploiement manuel :**

Configuration des ressources directement via l'interface du fournisseur.

Avantages : contrôle précis, simple pour petites infrastructures.

Limites : long, sujet aux erreurs, difficile à reproduire.

# Déploiement des infrastructures cloud

## **Déploiement automatisé**

- Utilisation d'outils comme Terraform ou Ansible pour décrire l'infrastructure en code.
- Avantages : rapide, reproductible, réduit les erreurs, facile à gérer à grande échelle.
- Limites : apprentissage nécessaire, configuration initiale complexe.



# Comparison entre Déploiement manuel et Déploiement automatisé

| Critère          | Déploiement manuel     | Déploiement automatisé             |
|------------------|------------------------|------------------------------------|
| Méthode          | Configuration manuelle | Infrastructure définie par du code |
| Temps            | Long                   | Rapide                             |
| Risque d'erreurs | Élevé                  | Faible                             |
| Reproductibilité | Difficile              | Élevée                             |
| Maintenance      | Complexe               | Simplifiée                         |
| Scalabilité      | Limitée                | Élevée                             |



## Comparison du temps de déploiement et du taux d'erreur pour l'infrastructure multi-cloud

Déploiement Manuel

4 Heures

Avec Terraform

10 Minutes

Risque d'Erreur (Manuel)

Élevé (Configuration, Sécurité)

Risque d'Erreur (IaC)

faible



## Infrastructure as Code (IaC)

---

L'Infrastructure as Code (IaC) est une méthode qui permet de gérer et déployer les infrastructures informatiques à l'aide de fichiers de configuration, comme du code, au lieu de les configurer manuellement.

### **Objectifs :**

- Automatiser les déploiements
- Réduire les erreurs humaines
- Assurer la cohérence des environnements

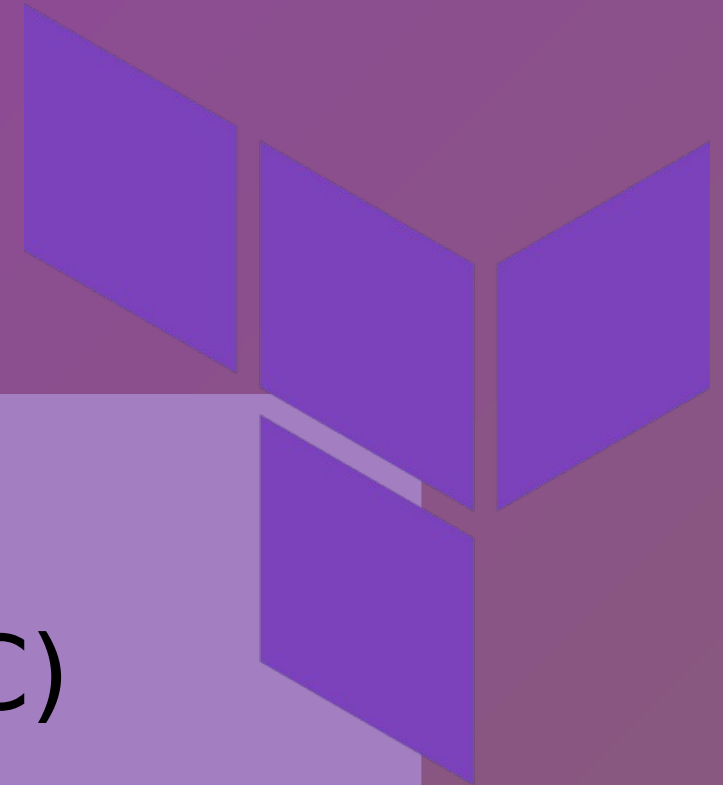
### **Avantages :**

- Déploiements rapides et fiables
  - Infrastructures reproductibles
  - Gain de temps
  - Maintenance simplifiée
- 

# TERRAFOR

## M

- Terraform est un outil d'Infrastructure as Code (IaC) développé par HashiCorp.
- Il permet de créer, modifier et gérer des infrastructures cloud de manière automatisée et déclarative.



# TERRAFORM

M

## Rôle de Terraform :

- Déployer des ressources cloud (serveurs, bases de données, réseaux...) automatiquement.
- Gérer et versionner l'infrastructure comme du code, facilitant les mises à jour et la maintenance.
- Assurer la cohérence et la reproductibilité des environnements.



# TERRAFOR

M

## **Pourquoi utiliser Terraform :**

- Gain de temps grâce à l'automatisation des déploiements.
- Réduction des erreurs humaines liées à la configuration manuelle.
- Compatibilité avec plusieurs fournisseurs cloud (AWS, Azure, GCP...).
- Possibilité de collaborer facilement grâce au versioning du code d'infrastructure.



# ARCHITECTURE ET FONCTIONNEMENT DE TERRAFORM

---

## **Fichiers de configuration :**

- Fichiers écrits en HCL avec l'extension .tf
- Décrivent l'infrastructure souhaitée
- Contiennent les providers, ressources et paramètres

## **Providers :**

- Permettent à Terraform de communiquer avec les plateformes cloud
- Chaque fournisseur cloud a son provider (AWS, Azure, GCP)
- Traduisent le code Terraform en appels API

# ARCHITECTURE ET FONCTIONNEMENT DE TERRAFORM

---

## **Ressources :**

- Représentent les éléments de l'infrastructure
- Exemples : machines virtuelles, réseaux, stockage
- Créées et gérées automatiquement par Terraform

## **Fichier d'état (State) :**

- Fichier qui stocke l'état réel de l'infrastructure
- Permet à Terraform de suivre les ressources existantes
- Assure la synchronisation entre le code et l'infrastructure

## Cycle de vie d'un déploiement avec Terraform

- **Initialisation de Terraform** (terraform init) :

Cette commande permet d'initialiser le projet Terraform. Elle télécharge les providers nécessaires et prépare l'environnement de travail.

- **Planification des changements** (terraform plan) :

Cette étape permet de visualiser les ressources qui vont être créées, modifiées ou supprimées, sans appliquer de changements réels.

## Cycle de vie d'un déploiement avec Terraform

- **Application du déploiement** (*terraform apply*) :

Cette commande applique les changements et crée automatiquement l'infrastructure définie dans les fichiers Terraform.

- **Vérification de l'infrastructure déployée :**

La vérification se fait à l'aide de commandes comme *terraform show* ou *terraform state list*, ou directement via l'interface du fournisseur cloud, afin de s'assurer que les ressources ont bien été déployées.

# Automatisation des déploiements avec Terraform

---

## Cycle de vie d'un déploiement avec Terraform





## **Automatisation :**

L'infrastructure est définie sous forme de code

Déploiement sans configuration manuelle

Réduction des erreurs humaines

Gain de temps et fiabilité

## **Scalabilité :**

Adaptation facile des ressources

Ajout ou suppression de ressources rapidement

Scalabilité verticale et horizontale

Optimisation des coûts selon les besoins

# ETUDE DE CAS

## Script PRATIQUE

setup.sh

```
GNU nano 2.9.2 scripts/setup.sh *
#!/bin/bash

echo "=== Mise à jour du système Linux ==="
sudo apt update
sudo apt upgrade -y

echo "=== Installation des outils nécessaires ==="
sudo apt install -y curl unzip git

echo "=== Vérification des outils ==="
git --version
curl --version

echo "=== Environnement prêt ==="
```

# ETUDE DE CAS

## Script PRATIQUE

```
GNU nano 2.9.2 deploy.sh scripts/deploy.sh *
#!/bin/bash

echo "=== Déploiement avec Terraform ==="

cd terraform || exit

terraform init
terraform validate
terraform fmt
terraform plan
terraform apply -auto-approve

echo "=== Déploiement terminé ==="

```

# ETUDE DE CAS PRATIQUE

```
omaima@omaima-ubuntu:~/terraform-multicloud$ ./scripts/deploy.sh  
=== Déploiement avec Terraform ===
```

```
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

Outputs:

```
aws_vm_file = "aws_vm.txt"  
azure_vm_file = "azure_vm.txt"
```

# ETUDE DE CAS

## PRATIQUE

Script **destroy.sh**

```
GNU nano 7.2 scripts/destroy
#!/bin/bash

echo "=== Suppression de l'infrastructure ==="

cd terraform || exit

terraform destroy -auto-approve
Aide
echo "=== Infrastructure supprimée ==="

```



## CONCLUSION

Ce projet a permis de comprendre l'automatisation des déploiements multi-cloud à l'aide de Terraform.

Cet outil simplifie la gestion des infrastructures, réduit les erreurs humaines et améliore la fiabilité des déploiements. Des améliorations futures peuvent être envisagées, notamment en matière de sécurité, d'intégration CI/CD et d'utilisation de services cloud plus avancés.





**M E R C I D E  
V O T R E  
A T T E N T I O N**